SEMANA 14 · INTELIGENCIA ARTIFICIAL EN LA NUBE I

IA en la Nube I: Modelos, Tokens e Inferencia

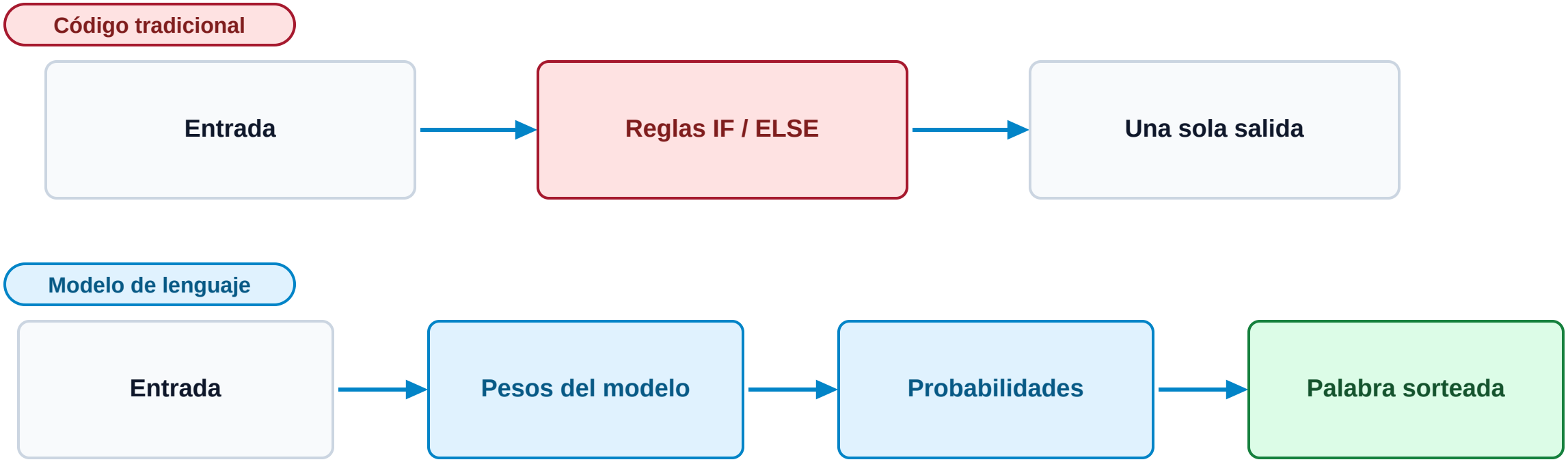
Parte 1 de 4 · Cómo un modelo genera texto y cuánta memoria exige

Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas
Semana 14 · Periodo 2026-2 · Barranquilla, Colombia

Parte 1 de 4 · Cómo un modelo genera texto y cuánta memoria exige

Del software determinista al probabilístico

Reglas escritas a mano frente a probabilidades aprendidas

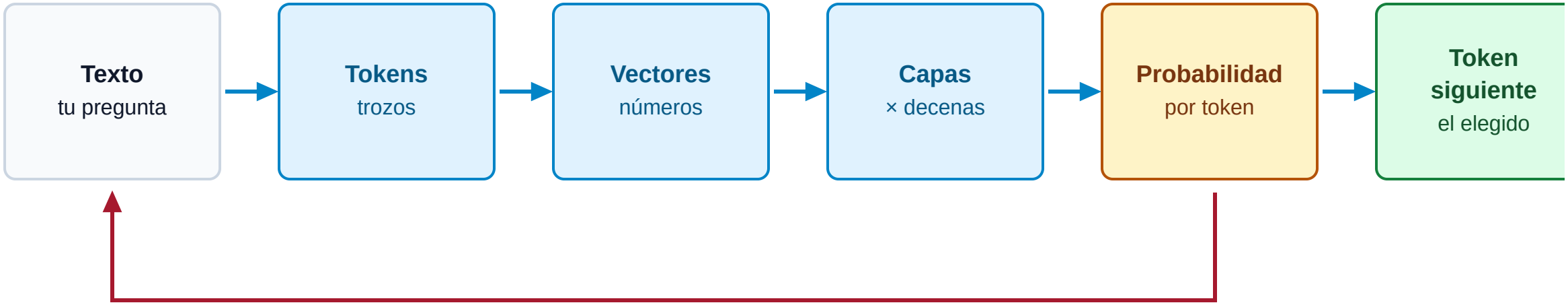


Misma pregunta, respuestas que pueden variar

Fuente: AWS Well-Architected, Generative AI Lens (docs.aws.amazon.com/wellarchitected/latest/generative-ai-lens)

Un modelo de lenguaje por dentro

Del texto de entrada al token de salida



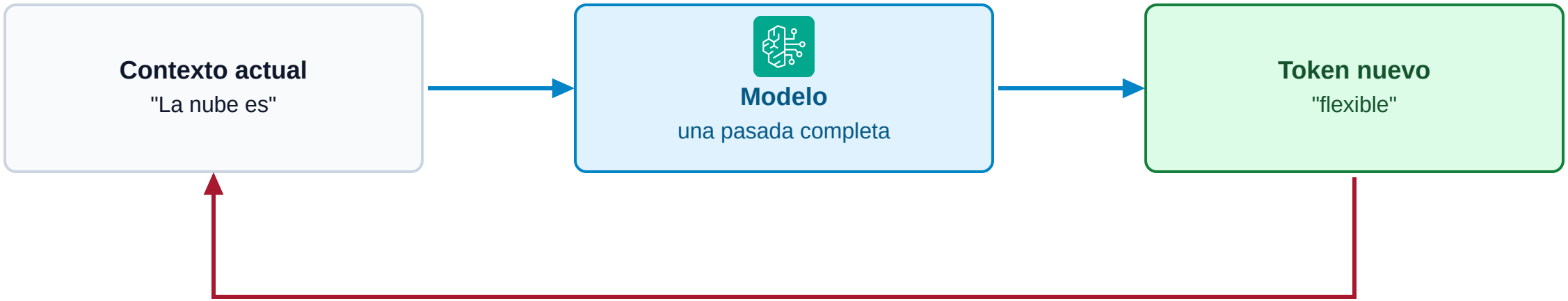
Se repite por cada token de la respuesta

Un modelo es un archivo de pesos + un programa que lo ejecuta

Fuente: Amazon Bedrock User Guide, Models at a glance (docs.aws.amazon.com/bedrock/latest/userguide/model-cards.html)

Un token a la vez

El modelo no escribe la respuesta de golpe



El token nuevo se agrega al contexto y el ciclo continúa

Respuesta larga = muchas pasadas por el modelo

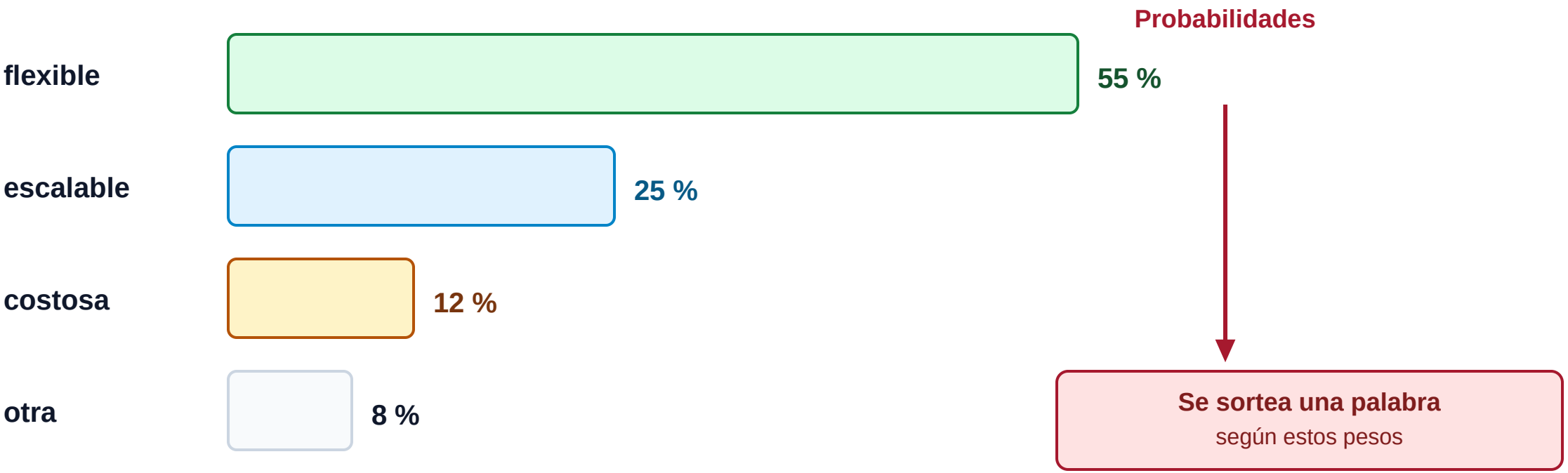
Fuente: Amazon Bedrock User Guide, Inference parameters (docs.aws.amazon.com/bedrock/latest/userguide/inference-parameters.html)

Cada paso es una apuesta ponderada

El modelo reparte 100 % entre las palabras posibles

La nube es ___

ejemplo ilustrativo

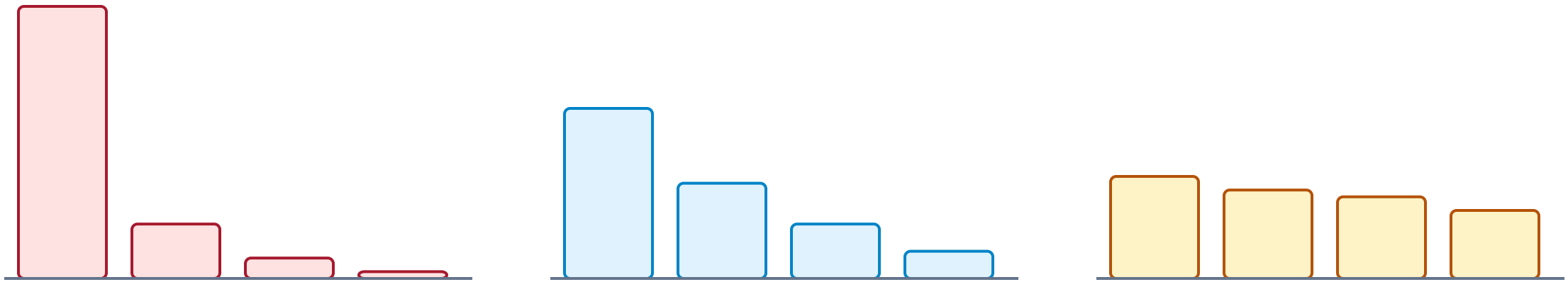


Fuente: Amazon Bedrock User Guide, Inference parameters (consulta 2026-11-05); valores ilustrativos

Temperatura: afila o aplana la curva

Un solo parámetro cambia el carácter de la respuesta

Temperatura baja	Temperatura media	Temperatura alta



Estable y repetible

SQL, extracción de datos

Natural

Chat de atención

Creativa, riesgo de errores

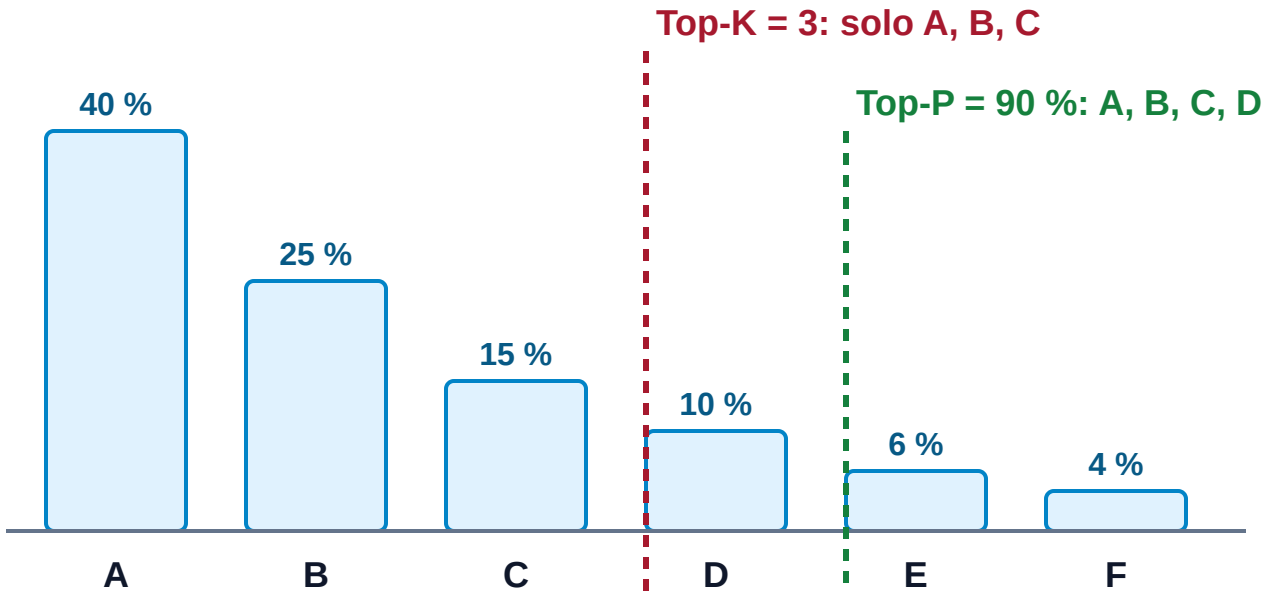
Lluvia de ideas

Parámetro "temperature" de la API: aplana o afila las probabilidades

Fuente: Amazon Bedrock User Guide, Inference parameters (docs.aws.amazon.com/bedrock/latest/userguide/inference-parameters.html)

Top-K y Top-P: dónde cortar la lista

Dos formas de descartar candidatos improbables



Top-K
corta por cantidad fija

Top-P
corta por probabilidad acumulada

Las palabras fuera del corte nunca se sortean

Fuente: Amazon Bedrock User Guide, Inference parameters (consulta 2026-11-05); valores ilustrativos

Entrenar un modelo frente a servirlo

Para la mayoría de equipos el reto es servir

Entrenamiento: una vez



Clúster grande
muchas GPU juntas

Archivo de pesos
el modelo ya aprendido

se copian

Inferencia: cada petición



Servidor con GPU
lee los pesos

Respuesta
token a token

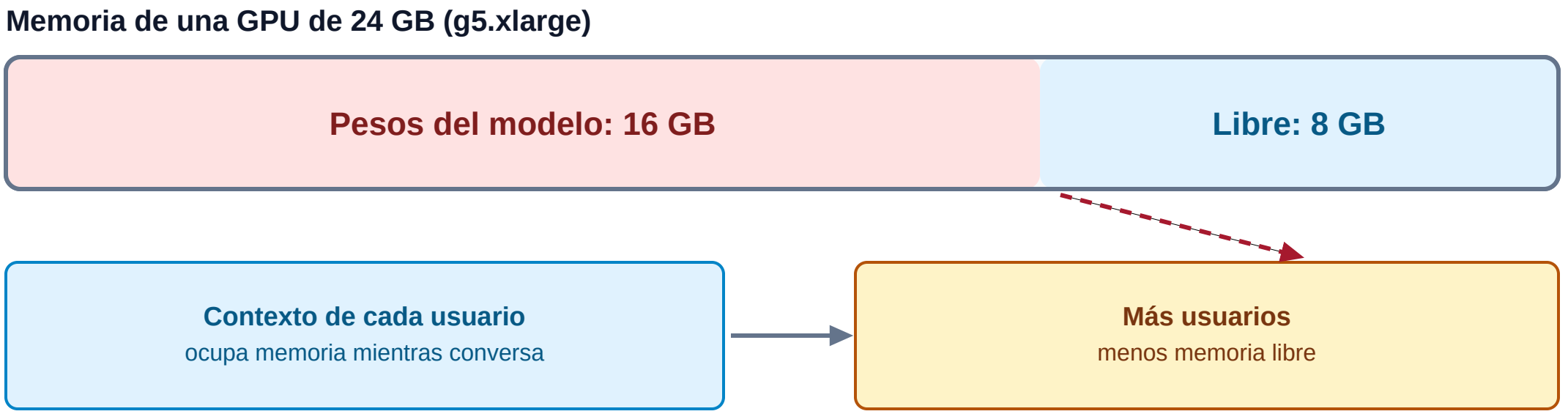
Se repite 24/7
costo continuo

Entrenar es un proyecto puntual; servir es una operación permanente

Fuente: AWS Well-Architected, Generative AI Lens (docs.aws.amazon.com/wellarchitected/latest/generative-ai-lens)

Los pesos ocupan la memoria primero

Lo que sobra atiende a los usuarios

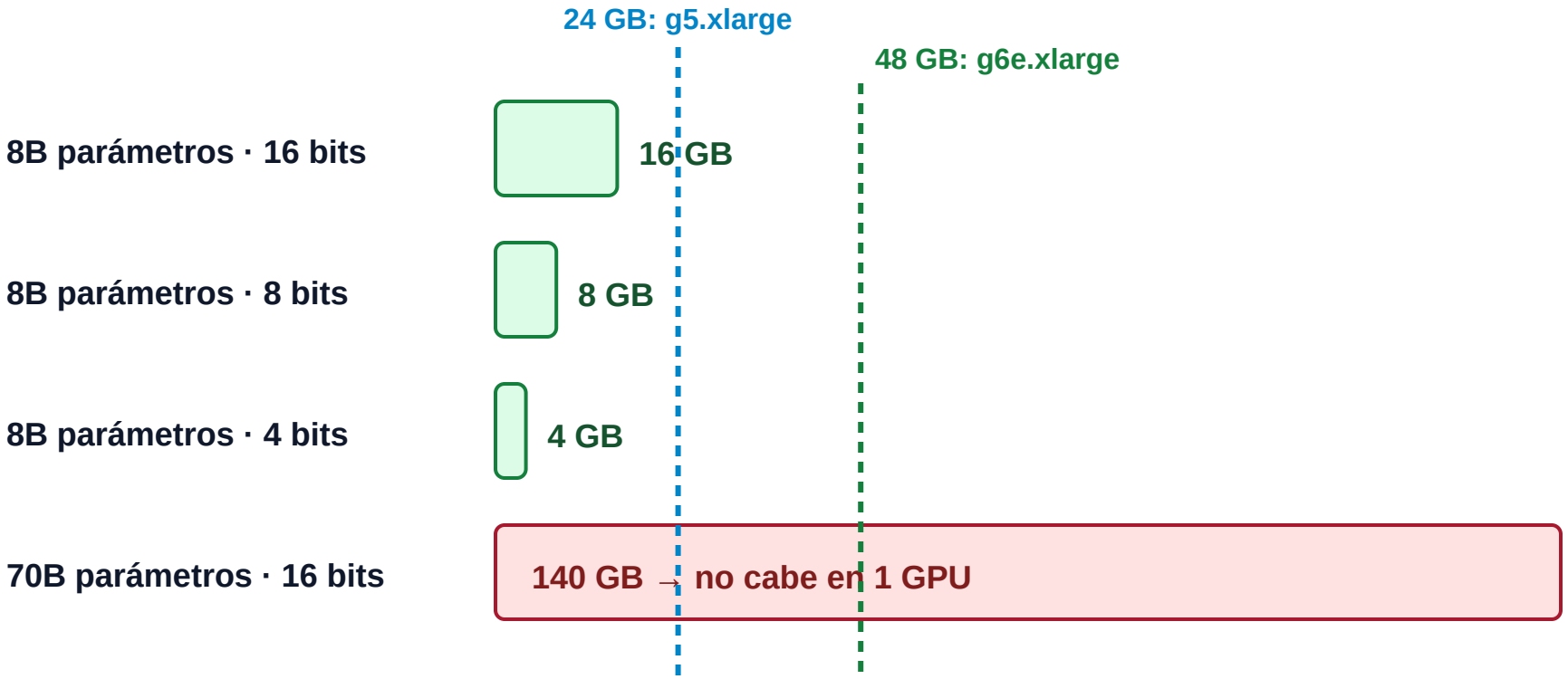


Modelo de 8 000 millones de parámetros en 16 bits = 16 GB de pesos

Fuente: AWS Price List, EC2 us-east-1 (g5.xlarge: 24 GB de GPU), consulta 2026-11-05; cálculo propio

Cuánta memoria necesita un modelo

Menos bits por parámetro, menos GB de pesos

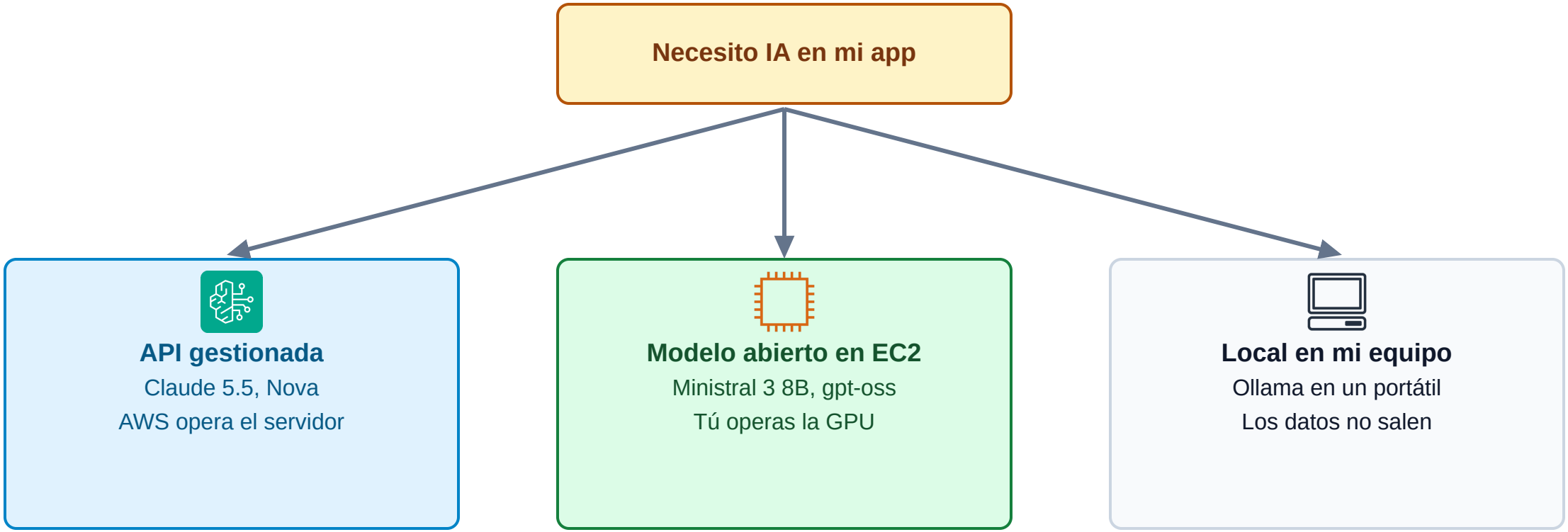


Pesos (GB) ≈ miles de millones de parámetros × bytes por parámetro

Fuente: AWS Price List EC2 us-east-1 (GPU de g5.xlarge y g6e.xlarge), consulta 2026-11-05; cálculo propio

Tres formas de usar un modelo

Quién opera el modelo define el costo y el riesgo

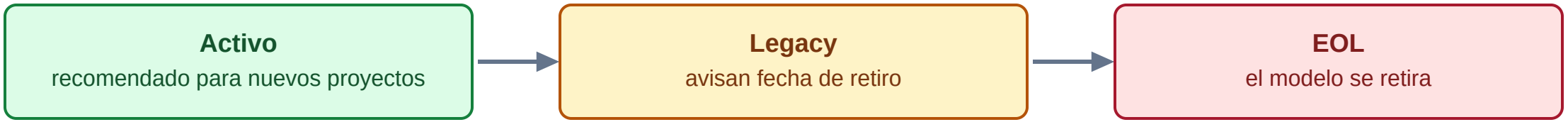


Más control = más operación a tu cargo

Fuente: Amazon Bedrock, Models at a glance (consulta 2026-11-05); Ollama (github.com/ollama/ollama)

Los modelos nacen, envejecen y se retiran

Datos consultados el 2026-11-05 en documentación de AWS



Ejemplos de la tabla oficial (histórico)



Activo hoy: Claude Sonnet 5.5

Diseña para cambiar de modelo

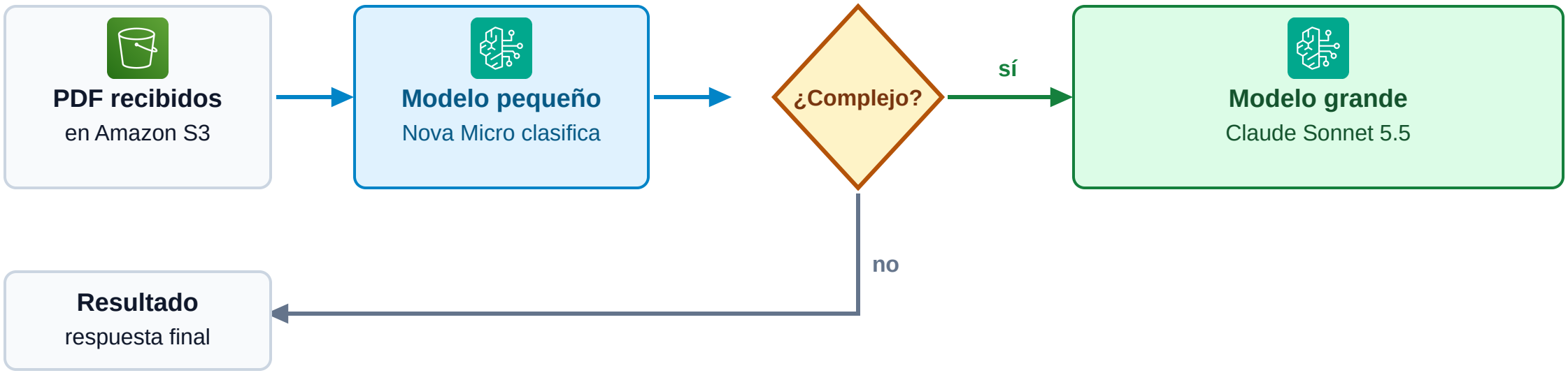
Fuente: docs.aws.amazon.com/bedrock/latest/userguide/model-lifecycle-legacy.html y model-cards.html, consulta 2026-11-05

PATRÓN DE CASCADA

Un modelo pequeño filtra, uno grande resuelve

Caso ilustrativo de clasificación documental

Caso ilustrativo de arquitectura, no es una empresa real

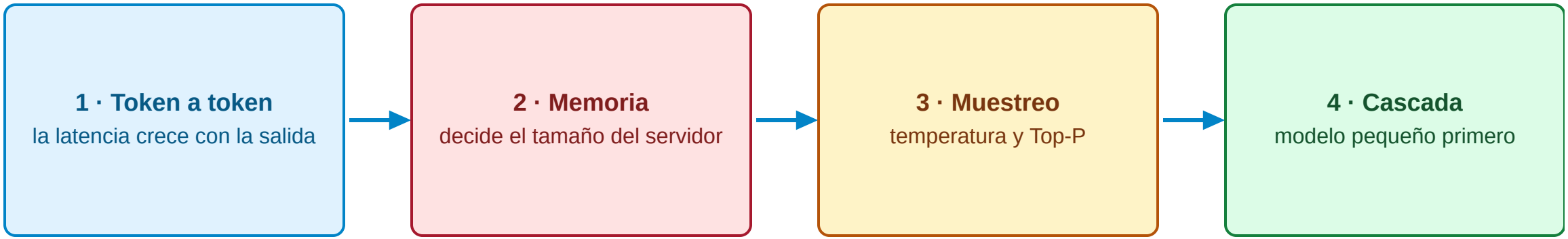


Lo simple lo resuelve el modelo barato; solo lo difícil sube

Fuente: Amazon Bedrock, Models at a glance y Pricing (consulta 2026-11-05); caso ilustrativo sin cifras de costo

Cuatro ideas para llevarse

Del mecanismo a la decisión de arquitectura

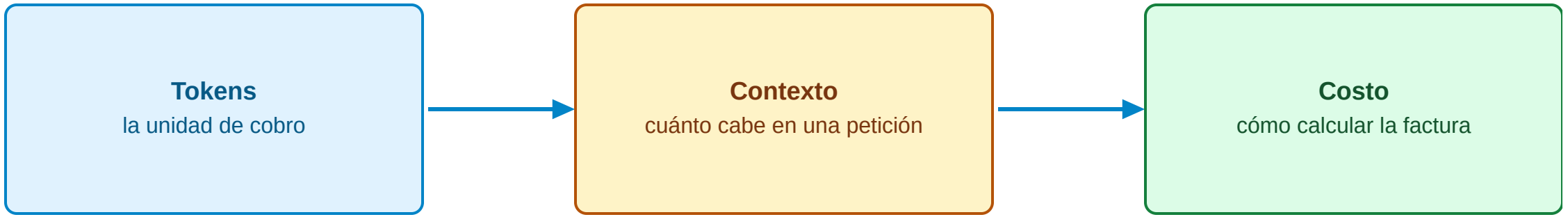


Cuatro decisiones que se repetirán en el resto de la clase

Fuente: síntesis de las diapositivas 2 a 13 de esta parte

Siguiente: tokens, contexto y costo

De lo que el modelo hace a lo que se factura



Parte 2: cuánto cuesta y cómo medirlo

SEMANA 14 · INTELIGENCIA ARTIFICIAL EN LA NUBE I

IA en la Nube I: Modelos, Tokens e Inferencia

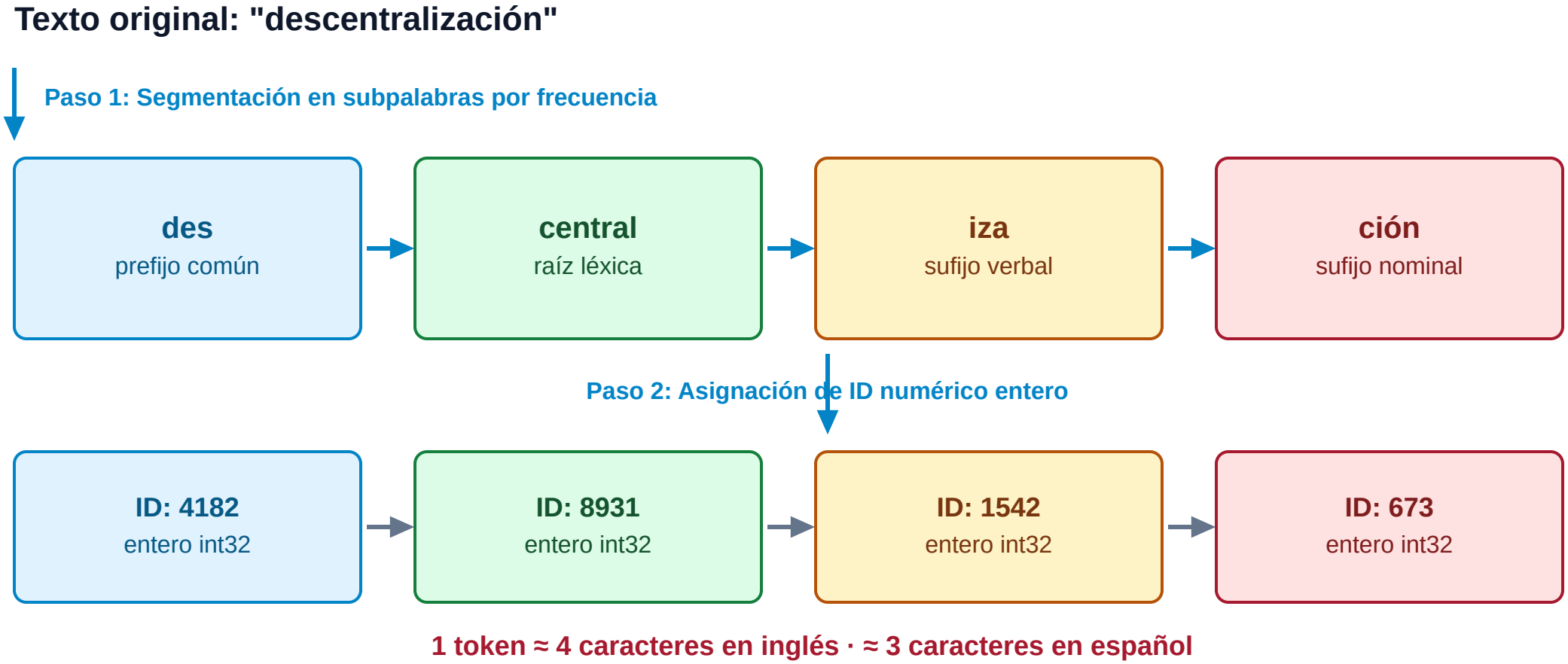
Parte 2 de 4 · Tokens, Ventana de Contexto y Economía Cloud

Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas
Semana 14 · Periodo 2026-2 · Barranquilla, Colombia

Parte 2 de 4 · Tokens, Ventana de Contexto y Economía Cloud

De caracteres a subpalabras (Tokens)

Cómo el texto se convierte en identificadores numéricos

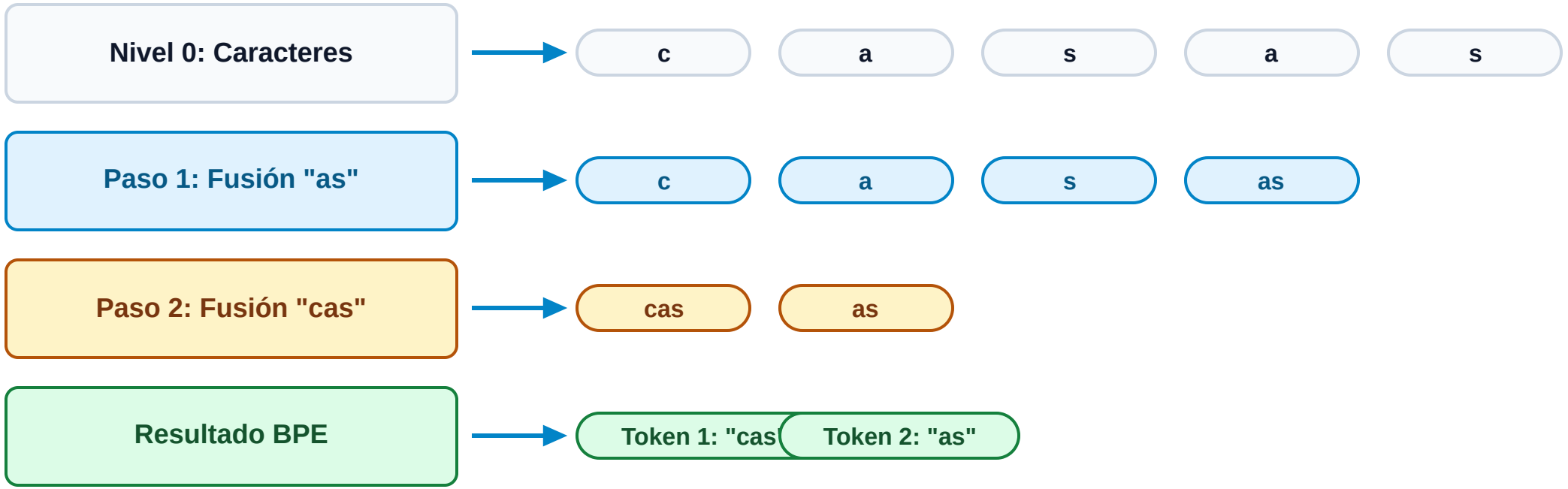


Fuente: Byte-Pair Encoding standard · OpenAI & Anthropic Tokenizer Documentation

Byte-Pair Encoding (BPE)

Fusión jerárquica de pares frecuentes hasta alcanzar el límite

Mecanismo iterativo de fusión por pares más frecuentes



Construye vocabulario cerrado sin palabras desconocidas (cero <UNK>)

Fuente: Sennrich et al. (2016) Neural Machine Translation of Rare Words with Subword Units

El impuesto de tokenización en español

Por qué el mismo prompt consume más tokens y cuesta más

Misma instrucción técnica evaluada en ambos idiomas

Inglés: "Deploy a private VPC"
Deploy (1) · a (1) · private (1) · VPC (1)
Total: 4 tokens

+75 %

Español: "Despliega una VPC privada"
Des (1) · plie (1) · ga (1) · una (1) · VPC (1) · priv (1) · ada (1)
Total: 7 tokens

Base: 1.00x

Sobrecosto léxico: +75 % tokens

Impacto Arquitectónico y FinOps en Nube

Mayor latencia TTFT y TPOT en español debido a más ciclos de inferencia

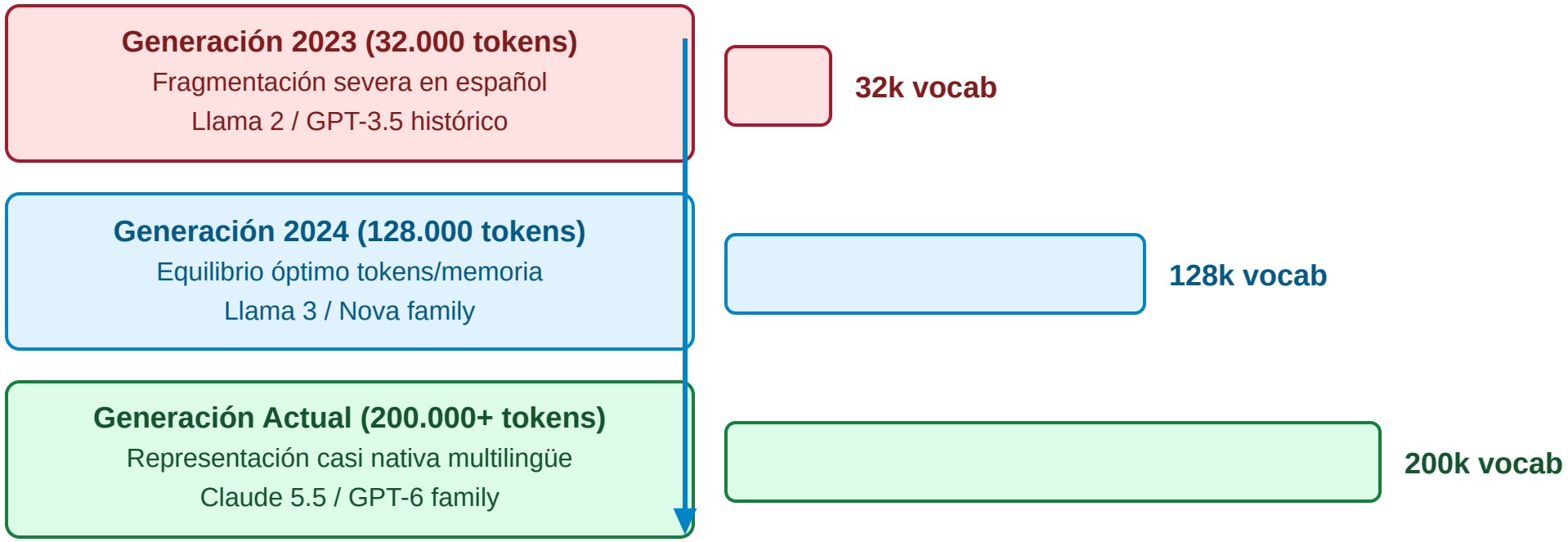
Presupuestos de IA en Colombia deben considerar entre 25 % y 50 % de sobrecosto por tokenización

Vocabularios modernos más amplios (128k+) reducen esta penalización

Fuente: Benchmarks comparativos de tokenizadores BPE · Petrov et al. (2023)

Vocabularios: de 32.000 a 200.000 tokens

Menos fragmentación a costa de mayor matriz de proyección en GPU



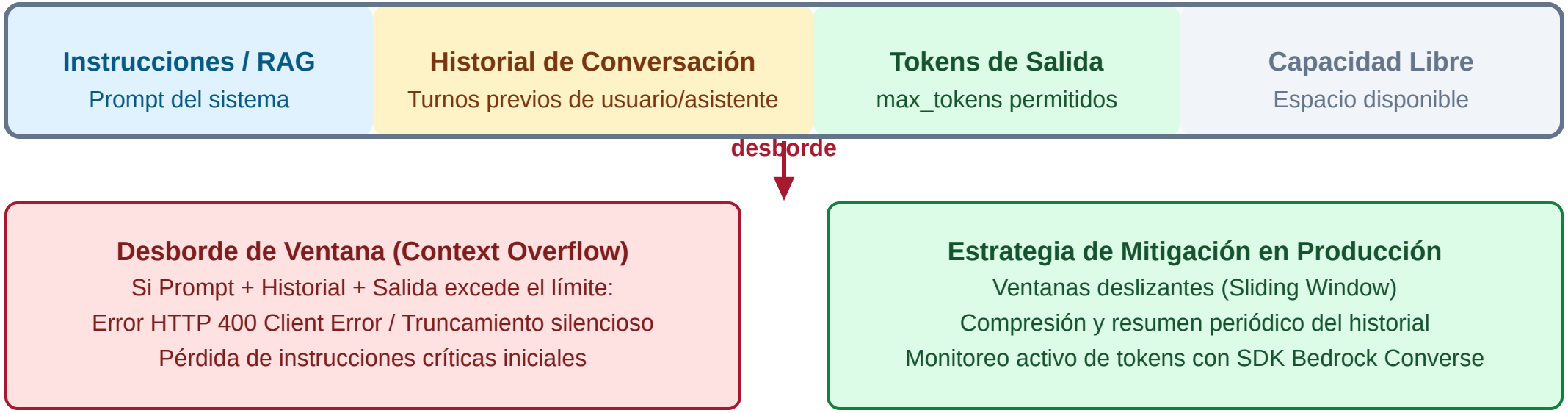
Compensación: Vocabularios gigantes consumen más VRAM en la capa de salida (LM Head)

Fuente: Model Cards oficiales · Anthropic, Meta y AWS Bedrock (2026)

Anatomía de la ventana de contexto

Cómo se reparte el espacio disponible y qué ocurre al desbordarse

Capacidad máxima de memoria temporal por petición (Context Window)



La ventana no es infinita: cada token adicional incrementa el costo y la latencia TTFT

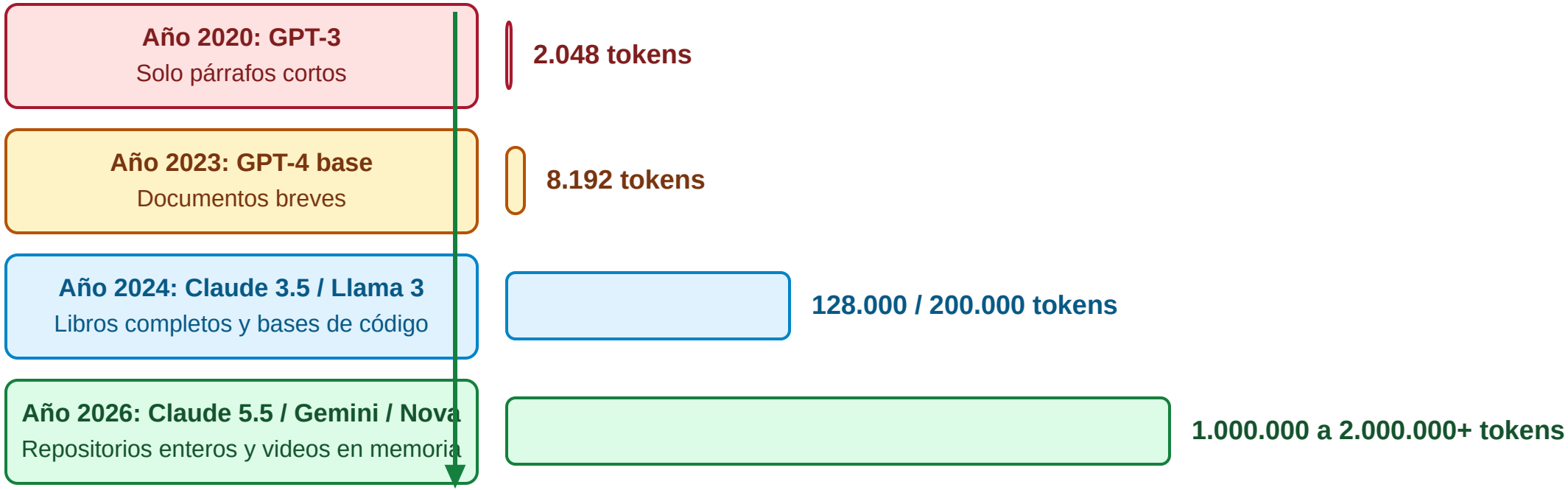
Fuente: AWS Bedrock Converse API Documentation · Quotas and Limits Guide

EVOLUCIÓN TEMPORAL

De 2.000 a más de 1.000.000 de tokens

La expansión del contexto transforma las arquitecturas de datos

Capacidad de contexto en modelos fundacionales (2020 - 2026)



Reto arquitectónico: Contextos masivos provocan degradación de atención ("Needle in a Haystack")

Fuente: Anthropic Context Window Specifications · Google DeepMind & OpenAI technical reports

Complejidad cuadrática y KV Cache

Por qué procesar contexto largo consume gigabytes masivos de GPU

Mecanismo de atención: Crecimiento cuadrático O(N²) de operaciones

Matriz NxN Tokens



N = 100k → 10^{10} pares

Solución Arquitectónica: KV Cache en GPU VRAM

1. Cada token procesado genera vectores Key y Value permanentes
2. En el paso siguiente se reutilizan los vectores previos sin recalcular
3. Costo de memoria: El KV Cache crece linealmente con cada nuevo token
4. En contextos de 100k+ tokens, el KV Cache puede superar el peso del propio modelo

FlashAttention optimiza el acceso a SRAM, reduciendo la saturación de ancho de banda

Fuente: Dao et al. (2022) FlashAttention · NVIDIA TensorRT-LLM Documentation

Prompt Caching: reutilizar en lugar de recalcular

Descuento drástico de costos y latencia en contextos compartidos

Mecanismo de Reutilización Inteligente de Contexto (Prompt Caching)

1. Primera Petición (Cache Write)

Prompt base: Manual de 20.000 tokens

GPU procesa y escribe KV Cache en memoria

Tarifa de entrada normal (100 %) · TTL: 5 minutos

reutiliza

2. Peticiones Siguientes (Cache Read)

Reutiliza el KV Cache existente sin recomputar

Descuento del 90 % al 95 % en tarifa de entrada

Latencia de primer token (TTFT) reducida hasta un 80 %

Regla de Diseño para Arquitectos Cloud

El prefijo compartido (instrucciones del sistema, esquemas SQL, documentos) debe ubicarse siempre al INICIO del prompt

Cualquier cambio al principio invalida la caché y obliga a un nuevo Cache Write

Disponible nativamente en Amazon Bedrock para Claude Sonnet 5.5, Haiku y Nova

Fuente: AWS Bedrock Prompt Caching Documentation (docs.aws.amazon.com/bedrock/latest/userguide/prompt-caching.html)

Entrada frente a Salida: la razón física del costo

Paralelismo eficiente en prefill frente a monopolio de memoria en decode

Por qué los tokens de salida cuestan entre 3x y 5x más que los de entrada



Tokens de Entrada (Prefill)

Procesamiento paralelo en bloques masivos

La GPU procesa miles de tokens simultáneamente

Uso intensivo de cómputo durante pocos milisegundos

Tarifa base reducida (ej. $2.00 / 1M tokens)

secuencia



Tokens de Salida (Decode)

Generación secuencial estricta (1 token por ciclo)

Monopoliza la memoria VRAM durante segundos

Limitado por ancho de banda de memoria (HBM bound)

Tarifa premium (ej. $10.00 / 1M tokens: 5x más)

Implicación FinOps: Controlar la verbosidad del modelo ahorra más que recortar el prompt

Configurar prompts para respuestas concisas y limitar max_tokens reduce el componente más costoso

El costo del token de salida refleja el tiempo exclusivo que el usuario retiene la GPU

Fuente: AWS Bedrock Pricing Model · FinOps Foundation GenAI Principles

Tarifas en Amazon Bedrock (Noviembre 2026)

Precios oficiales por millón de tokens en la región us-east-1

Tarifas Oficiales Amazon Bedrock · us-east-1 (Consulta en vivo: 2026-11-05)

Modelo Fundacional	Entrada / 1M	Salida / 1M	Perfil de Carga y Vigencia
Amazon Nova Micro	$0.035	$0.140	Texto ultrarrápido y clasificación
Amazon Nova Lite	$0.060	$0.240	Multimodal ligero balanceado
Amazon Nova Pro	$0.800	$3.200	Razonamiento avanzado propósito general
Claude Haiku 5.5	$0.100	$0.500	Alta velocidad y precisión (activo)
Claude Sonnet 5.5	$2.000	$10.000	Modelo de frontera para razonamiento y código
Claude Opus 5.5	$4.000	$20.000	Máxima capacidad cognitiva para tareas críticas
Claude 3.5 Sonnet (Histórico)	$3.000	$15.000	Estado: Legacy 10-oct-2026 · EOL 13-nov-2026

Fuente: aws.amazon.com/bedrock/pricing/ y endpoint oficial de tarificación meteredUnitMaps

Estimación de costos mensuales paso a paso

Comparativa financiera entre modelos medianos y de frontera

Ejercicio de dimensionamiento mensual: 50.000 consultas de soporte

Parámetros del Caso: 1.000 tokens de entrada (prompt) + 300 tokens de salida (respuesta) por consulta

Volumen mensual total: 50 Millones de tokens de entrada + 15 Millones de tokens de salida

Escenario A: Amazon Nova Pro

Tarifa: $0.80 entrada · $3.20 salida por 1M

Entrada: 50M × $0.80 = $40.00 USD

Salida: 15M × $3.20 = $48.00 USD

Costo Total Mensual: $88.00 USD

Escenario B: Claude Sonnet 5.5

Tarifa: $2.00 entrada · $10.00 salida por 1M

Entrada: 50M × $2.00 = $100.00 USD

Salida: 15M × $10.00 = $150.00 USD

Costo Total Mensual: $250.00 USD

Decisión FinOps: Enrutamiento Inteligente

Atender 80 % con Nova Pro ($70.40) + 20 % con Sonnet 5.5 ($50.00) = $120.40 USD mensuales

Ahorro del 52 % frente a usar exclusivamente el modelo superior, manteniendo máxima calidad

El costo de la IA en la nube se modela multiplicando volúmenes por tarifas oficiales unitarias

Fuente: Tarifas vigentes Amazon Bedrock us-east-1 (consulta 2026-11-05); cálculo matemático de arquitectura

Impacto financiero del Prompt Caching

Cómo la persistencia en memoria reduce la factura en más de un 80 %

Ahorro mensual incorporando Prompt Caching en Claude Sonnet 5.5

Sin Prompt Caching (100 % Ingesta Completa)

50.000 consultas con prompt base de 2.000 tokens

Volumen entrada: 100 Millones de tokens / mes

Tarifa normal: $2.00 por 1M tokens

Costo de entrada mensual: $200.00 USD

-83 %

Con Prompt Caching (90 % Acierto en Memoria)

Cache Write (10 %): 10M × $2.50 = $25.00 USD

Cache Read (90 %): 90M × $0.10 = $9.00 USD

Costo de entrada mensual: $34.00 USD

Ahorro neto: $166.00 USD mensuales (83 % reducción)

Parámetros Oficiales en Bedrock (docs.aws.amazon.com/bedrock/latest/userguide/prompt-caching.html)

Claude Sonnet 5.5: Cache read a $0.10 / 1M (descuento del 95 % frente a $2.00 de entrada)

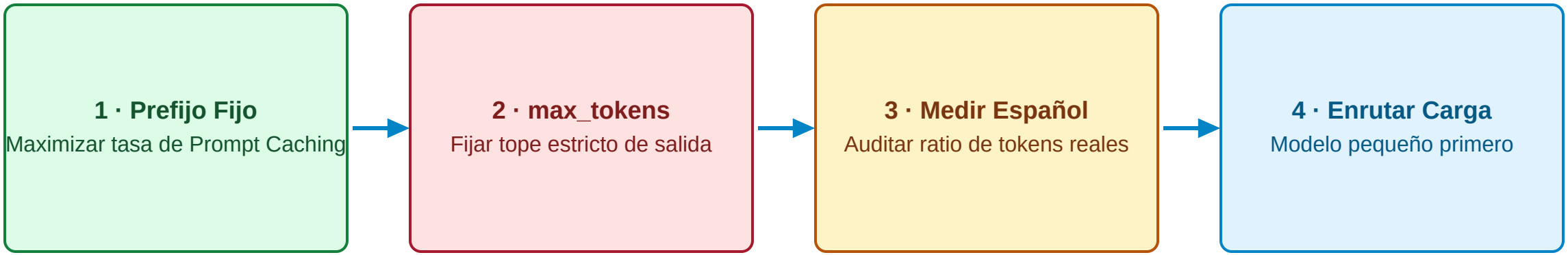
Válido para prefijos mayores a 1.024 tokens · Retención activa de 5 minutos en memoria de inferencia

El diseño del prompt determina si una aplicación es financieramente viable a escala

Fuente: AWS Bedrock Prompt Caching Pricing (tarifa cache read Sonnet 5.5: $0.10/1M, consulta 2026-11-05)

Cuatro reglas de oro en gestión de tokens

Directrices de ingeniería para controlar costos y latencias



Gobernanza y optimización sistemática del consumo de tokens en la nube

Fuente: AWS Generative AI Lens · Production Architectures Best Practices

Siguiente: el servidor de inferencia en la nube

Cómo se despliegan, monitorean y escalan los modelos en servidores reales



Parte 3: del modelo aislado al servidor de inferencia en producción

SEMANA 14 · INTELIGENCIA ARTIFICIAL EN LA NUBE I

IA en la Nube I: Modelos, Tokens e Inferencia

Parte 3 de 4 · El Servidor de Inferencia, Motores y Métricas de Rendimiento

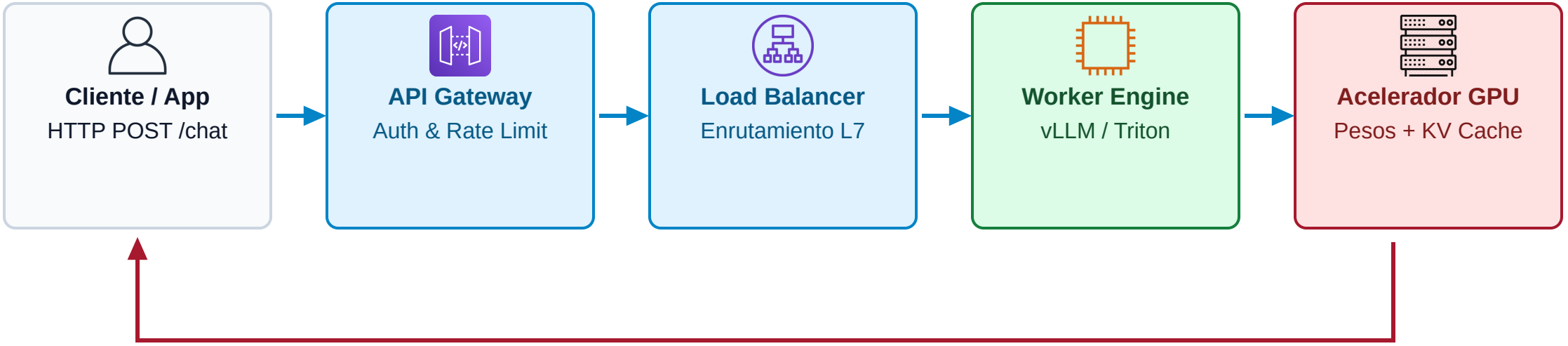
Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas
Semana 14 · Periodo 2026-2 · Barranquilla, Colombia

Parte 3 de 4 · El Servidor de Inferencia, Motores y Métricas de Rendimiento

Flujo integral de una petición de inferencia

Del cliente HTTP al acelerador de hardware en centro de datos

Ruta de extremo a extremo en infraestructura cloud



Flujo de respuesta streaming token a token (Server-Sent Events)

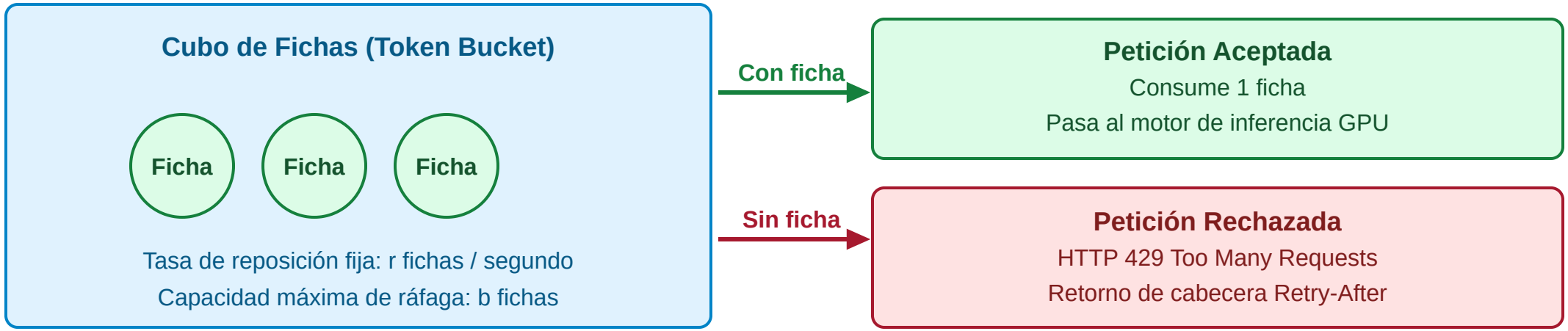
La separación de capas desacopla la autenticación del cómputo intensivo de GPU

Fuente: AWS Well-Architected Framework: Machine Learning Lens (Architecture Topologies)

Protección contra sobrecarga: Token Bucket

Cómo la capa perimetral evita la saturación de los aceleradores

Control perimetral de tráfico con algoritmo Token Bucket



Doble Cuota en Amazon Bedrock: RPM (Requests Per Minute) y TPM (Tokens Per Minute)

El cliente debe gestionar reintentos automáticos con retroceso exponencial (Exponential Backoff)

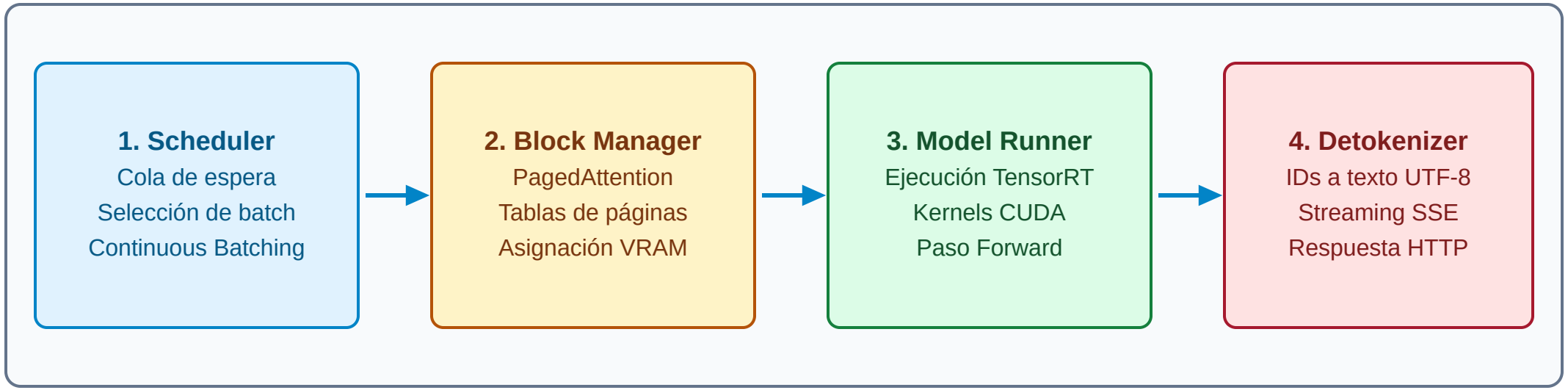
Evita el colapso por agotamiento de memoria VRAM en los aceleradores backend

Fuente: docs.aws.amazon.com/bedrock/latest/userguide/quotas.html (Cuotas oficiales de Bedrock)

Anatomía interna de un motor de inferencia

Los cuatro subsistemas que maximizan la utilización de la GPU

Componentes internos de un motor de alto rendimiento (ej. vLLM / SGLang)



Objetivo Central del Motor: Mantener los Tensor Cores saturados y la memoria sin fragmentación

El motor desacopla la recepción asíncrona de peticiones web del ciclo síncrono de cálculo en GPU

El software de inferencia es tan importante como el silicio de la GPU para lograr alto rendimiento

Fuente: Kwon et al. (2023) Efficient Memory Management for Large Language Model Serving with PagedAttention

Continuous Batching: cero tiempo ocioso en GPU

Cómo la agrupación a nivel de iteración multiplica el rendimiento

Batching Estático Tradicional frente a Continuous Batching por Iteración

Batch Tradicional (Estático)

Req 1: 50 tokens [==========]

Req 2: 10 tokens [====] TIEMPO MUERTO (GPU OCIOSA)

Req 3: 20 tokens [=======] TIEMPO MUERTO

Tiempo →

Continuous Batching (A nivel de iteración)

Req 1: 50 tokens [==========]

Req 2 finaliza → Entra Req 4 INMEDIATAMENTE

Req 3 finaliza → Entra Req 5 SIN ESPERAR

Ganancia de Rendimiento en Centro de Datos

El batch tradicional bloquea todo el grupo hasta que la petición más larga termina

Continuous Batching inserta y retira peticiones en cada ciclo individual de token

Incremento de Throughput de 2x a 4x con idéntico hardware de GPU

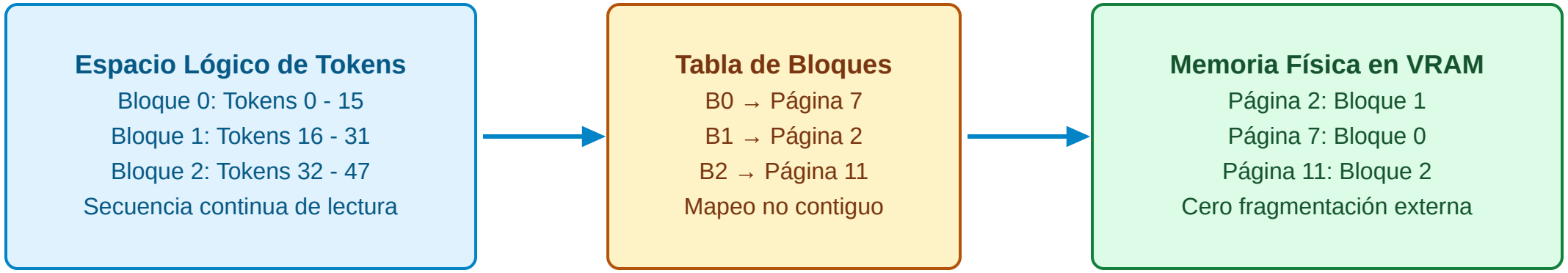
Estándar obligatorio implementado en vLLM, TensorRT-LLM y motores gestionados cloud

Fuente: Yu et al. (2022) Orca: A Distributed Serving System for Transformer-Based Generative Models

PagedAttention: sin fragmentación en VRAM

Traducción de bloques lógicos a físicos inspirada en sistemas operativos

Mecanismo de PagedAttention: Memoria virtual paginada para KV Cache



Impacto Arquitectónico Directo

Antes de PagedAttention: Se reservaba memoria estática para el contexto máximo (desperdicio > 60 % de VRAM)

Con PagedAttention: Asignación dinámica de páginas bajo demanda conforme crecen los tokens

Permite compartir bloques idénticos en memoria física (fundamental para prefijos de Prompt Caching)

Inspirado en la memoria virtual clásica de los sistemas operativos (Kwon et al., SOSP 2023)

Fuente: Kwon et al. (2023) vLLM Project Whitepaper · UC Berkeley

Las dos fases en acción: Prefill frente a Decode

De saturación de núcleos de cálculo a saturación de ancho de banda

Separación física de fases en la ejecución de un modelo

Fase 1: Prefill (Contexto Inicial)

Ingesta de todos los tokens del prompt simultáneamente

Cómputo masivo de matrices (Compute-Bound)

Satura núcleos Tensor de cómputo en paralelo

Genera el KV Cache inicial en memoria VRAM

Métrica que gobierna: TTFT (Time to First Token)

→

Fase 2: Decode (Generación Secuencial)

Produce estrictamente un token por cada iteración

Lectura constante de gigabytes de pesos y KV Cache

Limitado por ancho de banda de memoria (Memory-Bound)

Retiene la GPU hasta finalizar el texto completo

Métrica que gobierna: TPOT (Time Per Output Token)

Tendencia Avanzada: Desagregación Prefill/Decode

Servidores con chips orientados a cómputo para Prefill + Servidores con máximo ancho de banda para Decode

Evita que peticiones largas de entrada bloqueen la generación fluida de otros usuarios

Comprender este desacople es vital para interpretar correctamente los dashboards de telemetría

Fuente: Patel et al. (2024) Splitwise: Efficient Generative LLM Serving Using Phase Splitting

Métricas esenciales: TTFT, TPOT y Throughput

Los indicadores que definen el Acuerdo de Nivel de Servicio (SLA)

Las cuatro métricas canónicas de SLA en inferencia cloud

TTFT (Time to First Token)	**Milisegundos hasta recibir el primer token** Impacto crítico en percepción de latencia interactiva
TPOT (Time Per Output Token)	**Milisegundos por cada token sucesivo generado** Debe ser < 50 ms para superar la velocidad de lectura humana
Throughput (Tokens / Segundo)	**Volumen total agregado procesado por el clúster** Métrica reina para dimensionar capacidad y costos de flota
Latencia p99 de Cola	**Tiempo de espera en cola antes de tocar la GPU** Detecta cuellos de botella y necesidad de escalado horizontal

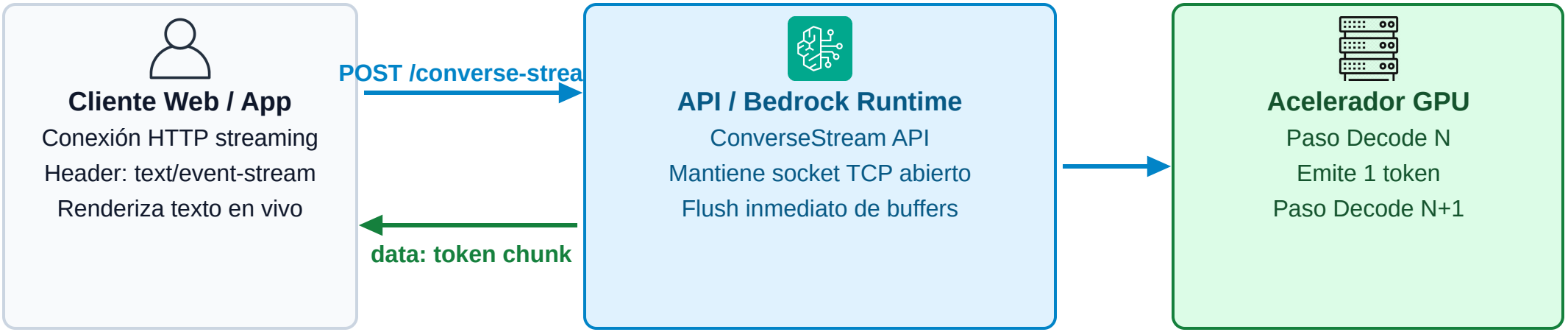
Monitoreadas en Amazon Bedrock mediante CloudWatch Metrics y Model Invocation Logging

Fuente: AWS Bedrock Monitoring Documentation (docs.aws.amazon.com/bedrock/latest/userguide/monitoring-cw.html)

Streaming con Server-Sent Events (SSE)

Por qué entregar token por token es obligatorio en arquitecturas modernas

Mecanismo de streaming HTTP mediante Server-Sent Events



Impacto en la Experiencia de Usuario (UX)

Sin Streaming: Usuario espera 8 segundos en blanco hasta que el modelo escribe 400 tokens completos (TTFT = 8s)

Con Streaming: El primer token aparece en 400 ms, transformando la percepción de respuesta del sistema

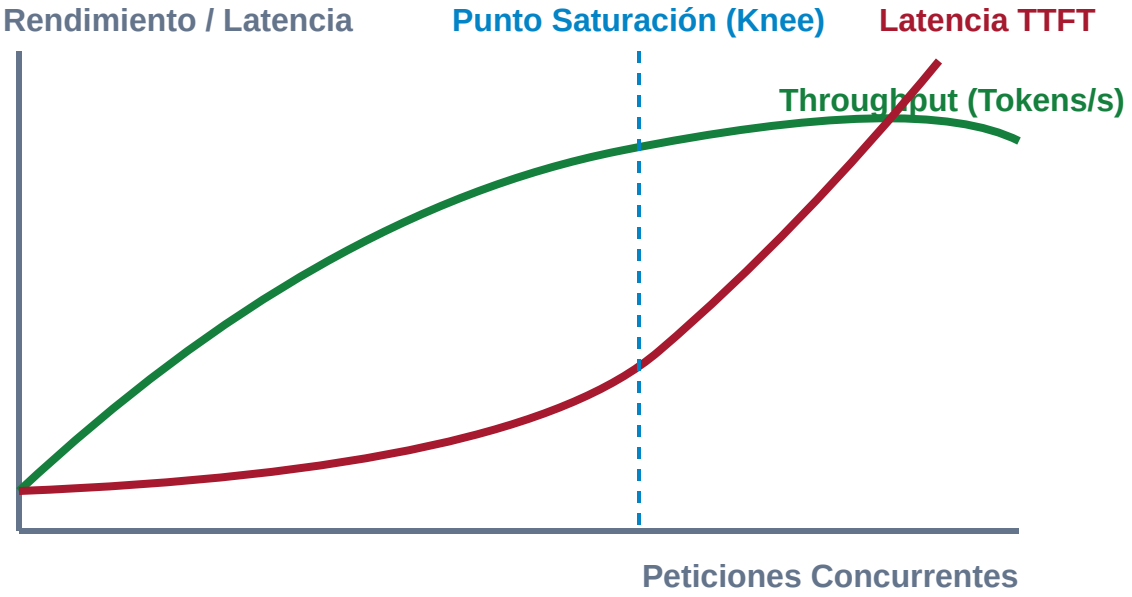
En arquitecturas modernas, el streaming sobre HTTP/2 es un requisito no negociable de diseño

Fuente: AWS Bedrock ConverseStream API Reference · W3C Server-Sent Events Standard

Curva de saturación: el codo de la concurrencia

Cómo encontrar el punto óptimo de operación antes de que la latencia colapse

Comportamiento de Throughput y Latencia bajo concurrencia creciente



Zona Óptima (Izquierda de la línea)

Alta utilización de núcleos Tensor

Latencia controlada dentro de SLA

Throughput crece casi linealmente

Zona de Saturación (Derecha de la línea)

Colas de espera en el Scheduler

La latencia explota exponencialmente

Throughput se estanca por falta de VRAM

Regla de Oro de Capacidad: El escalado horizontal debe dispararse ANTES de alcanzar el punto de saturación

Configurar alertas de concurrencia al 70 % del codo de saturación para absorber ráfagas de tráfico

El dimensionamiento empírico mediante pruebas de carga de estrés es obligatorio antes de pasar a producción

Fuente: Patel et al. (2024) LLM Serving Benchmarks · vLLM Performance Guides

Instancias GPU en AWS: especificaciones y costos

Tarifas reales por hora para servidores dedicados de inferencia

Aceleradores de IA en Amazon EC2 (Tarifas Oficiales us-east-1, Consulta: 2026-11-05)

Instancia EC2	Hardware GPU / Memoria	Tarifa On-Demand	Caso de Uso Recomendado
g5.xlarge	1x NVIDIA A10G (24 GB GDDR6)	$1.006 / hora	Modelos 8B cuantizados · Inferencia general
g5.12xlarge	4x NVIDIA A10G (96 GB Total)	$5.672 / hora	Modelos 70B en 4 bits · Concurrencia
g6.xlarge	1x NVIDIA L4 (24 GB GDDR6)	$0.805 / hora	Inferencia balanceada INT8/FP8
g6e.xlarge	1x NVIDIA L40S (48 GB GDDR6)	$1.861 / hora	Modelos 14B/32B sin cuantizar
g6e.12xlarge	4x NVIDIA L40S (192 GB Total)	$10.493 / hora	Modelos 70B nativos · Gran contexto
p5.48xlarge	8x NVIDIA H100 (640 GB HBM3)	$55.040 / hora	Modelos de frontera masivos

Fuente: AWS Price List API / EC2 us-east-1 ondemand index (versión 20261104210305, validada en vivo)

Auto Scaling en flotas de inferencia

Target Tracking y mitigación del arranque en frío (Cold Start)

Autoescalado dinámico mediante Target Tracking en Amazon EC2



El Desafío Crítico: Tiempo de Arranque en Frío (Cold Start)

Descargar 16 GB de pesos desde S3 y cargarlos en VRAM toma entre 2 y 5 minutos

Soluciones de arquitectura: Volúmenes EBS pre-calentados (Fast Snapshot Restore) o contenedores con pesos incrustados

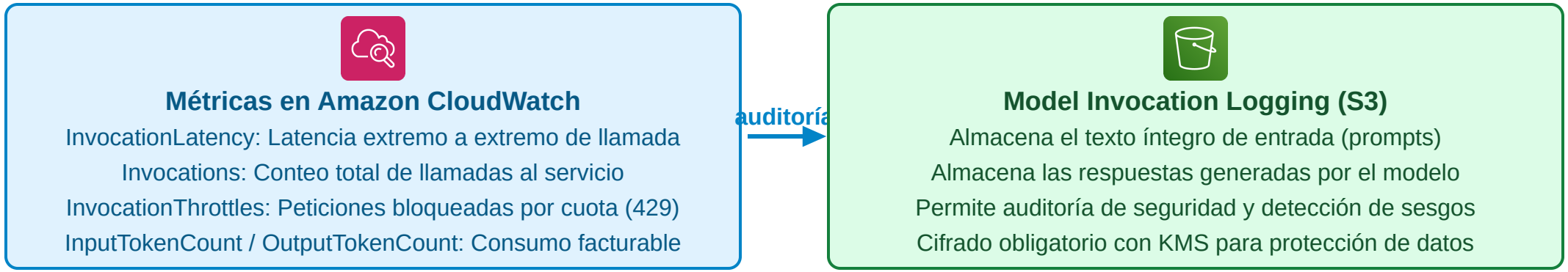
Escalar de forma proactiva basándose en tendencias horarias antes de que la cola sature el SLA

Fuente: docs.aws.amazon.com/autoscaling/ec2/userguide/as-scaling-target-tracking.html

Observabilidad: CloudWatch y registro de peticiones

Monitoreo de latencia, cuotas y auditoría integral de prompts en Amazon S3

Telemetría oficial en Amazon Bedrock (CloudWatch Metrics y Logs)



Alarma Recomendada de Arquitectura en CloudWatch

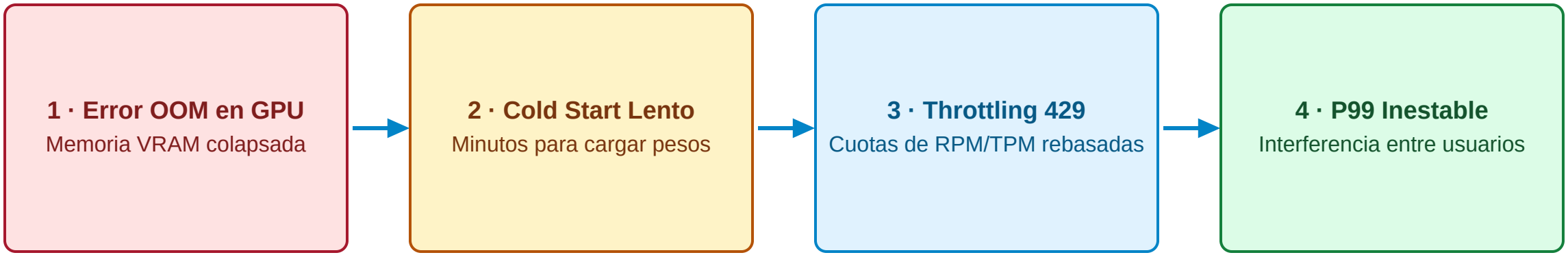
Disparar alarma si InvocationThrottles > 0 durante 5 minutos consecutivos

Acción automática: Notificar por SNS y solicitar incremento de cuota de tokens por minuto (TPM) en Service Quotas

Fuente: docs.aws.amazon.com/bedrock/latest/userguide/monitoring-cw.html y model-invocation-logging.html

Cuatro fallas clásicas en inferencia y cómo mitigarlas

De caídas por falta de memoria a picos de latencia en percentiles altos



Diagnóstico preventivo y mitigación de incidentes críticos en clústeres de IA

Fuente: AWS Well-Architected Framework: Machine Learning Lens (Reliability Pillar)

Siguiente: la gran decisión de arquitectura

Comparativa definitiva entre servicios gestionados, servidores en nube y modelos locales



Parte 4: criterios técnicos y regulatorios para tomar la decisión arquitectónica

SEMANA 14 · INTELIGENCIA ARTIFICIAL EN LA NUBE I

IA en la Nube I: Modelos, Tokens e Inferencia

Parte 4 de 4 · Decisiones de Despliegue, Seguridad y Marco Regulatorio

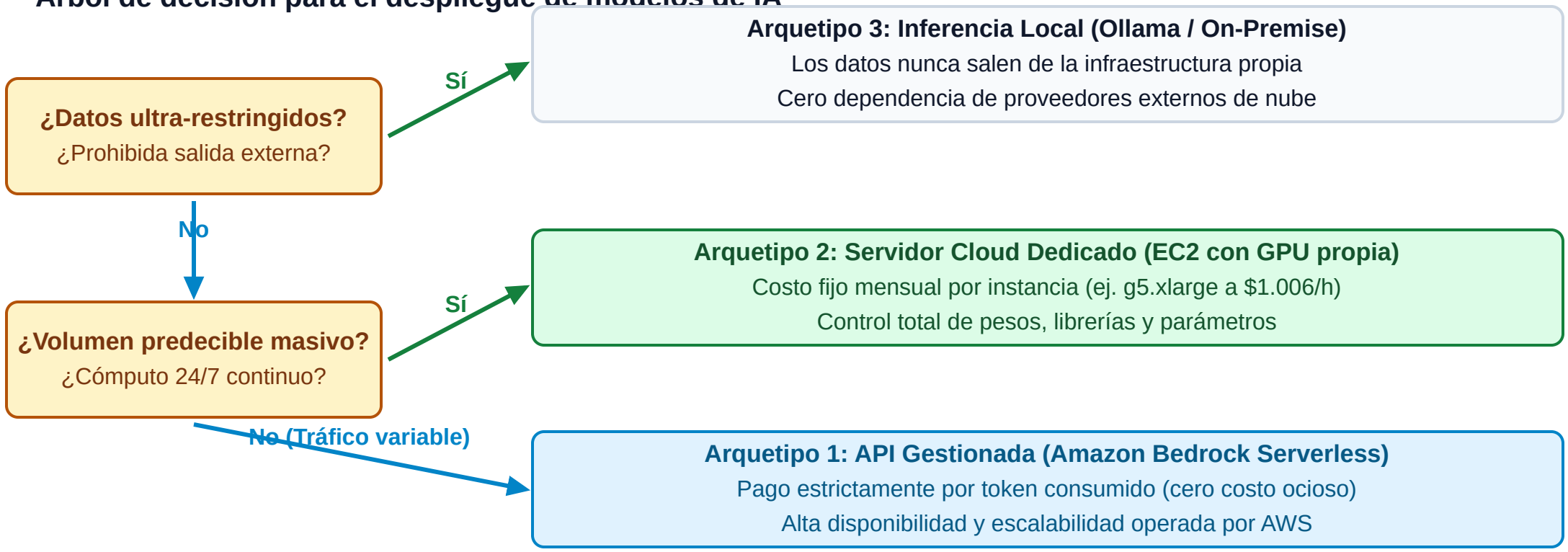
Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas
Semana 14 · Periodo 2026-2 · Barranquilla, Colombia

Parte 4 de 4 · Decisiones de Despliegue, Seguridad y Marco Regulatorio

Los tres caminos para desplegar IA

Árbol de decisión entre APIs gestionadas, servidores propios y entornos locales

Árbol de decisión para el despliegue de modelos de IA



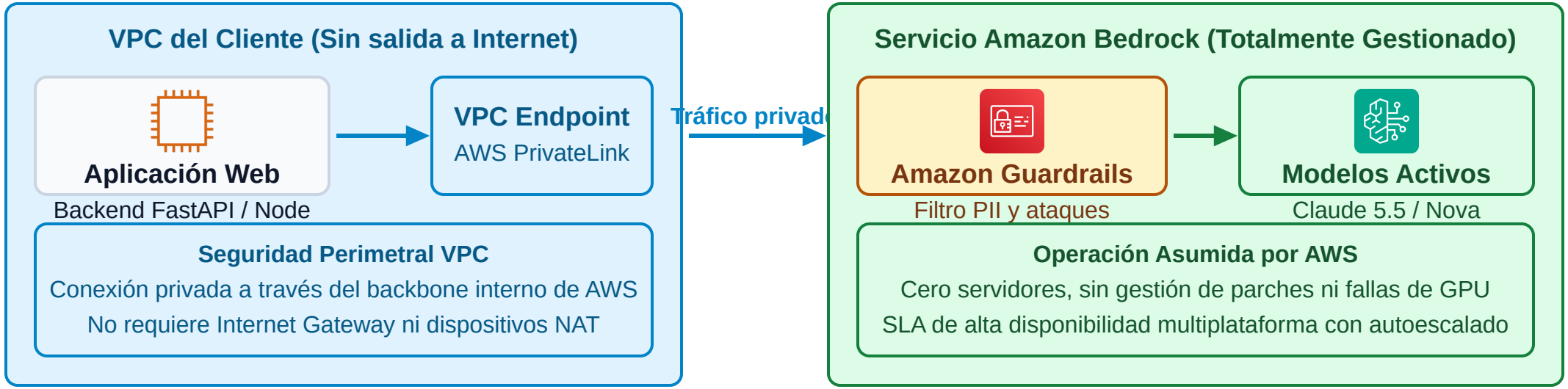
La elección técnica debe justificar costo financiero, esfuerzo operativo y cumplimiento legal

Fuente: AWS Decision Guide for Generative AI · Well-Architected Framework

Arquetipo 1: Inferencia serverless con Amazon Bedrock

Máxima velocidad de entrega, escalado elástico y aislamiento en red privada

Arquetipo 1: Consumo Serverless mediante Amazon Bedrock



Garantía Contractual: AWS no utiliza los datos ni prompts del cliente para reentrenar modelos base

Fuente: docs.aws.amazon.com/bedrock/latest/userguide/vpc-interface-endpoints.html

Fuente: AWS Bedrock Security and Architecture Documentation (PrivateLink and Endpoints Guide)

Pros y contras del enfoque gestionado

Velocidad de despliegue y cero costo ocioso frente a límites de cuota y control

Balance de compromisos en arquitecturas Serverless gestionadas

Ventajas Principales (Pros)

Cero costo fijo: Si nadie consulta, la factura es $0 USD

Tiempo a producción de horas en lugar de semanas

Acceso inmediato a modelos de frontera actualizados (Claude 5.5)

SLA gestionado y alta disponibilidad geográfica integrada

Herramientas nativas de seguridad (Guardrails, KMS, IAM)

vs

Límites y Desventajas (Contras)

Costo por token más alto a gran escala continua

Sujeto a cuotas de RPM y TPM (Throttling 429)

Cero control sobre los pesos o arquitectura de red interna

Dependencia de la disponibilidad de la API del proveedor

Riesgo de retiro de modelos (Lifecycle Legacy / EOL)

Veredicto de Arquitectura: Ideal para comenzar, validar productos y operar con tráfico variable

El 80 % de las aplicaciones empresariales modernas inician en este arquetipo por su óptima relación costo-beneficio

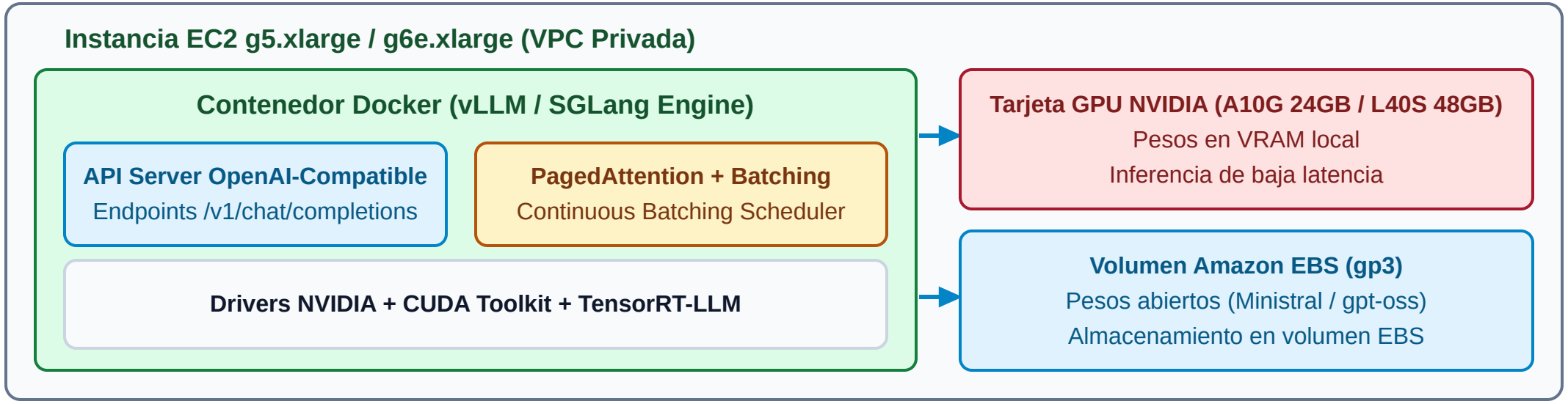
La agilidad para probar modelos nuevos compensa con creces el costo unitario de token en etapas tempranas

Fuente: AWS Well-Architected Framework: Machine Learning Lens (Trade-off Analysis)

Arquetipo 2: Servidor dedicado con GPU en EC2

Control absoluto de la pila tecnológica a cambio de costos fijos y gestión operativa

Arquetipo 2: Despliegue de Servidor Propio con GPU en Amazon EC2



Costo Fijo Continuo: Instancia g5.xlarge = $1.006 / hora ($734.38 USD mensuales en uso 24/7)

Fuente: AWS Price List API us-east-1 ondemand index (consulta oficial: 2026-11-05 UTC)

Requiere equipo de infraestructura dedicado para mantenimiento de SO, parches y fallas de hardware

Fuente: AWS EC2 G5 & G6e Instance Documentation · vLLM Production Deployment Guide

Pros y contras del servidor dedicado con GPU

Capacidad dedicada y costo marginal nulo frente a costos fijos y mantenimiento continuo

Balance de compromisos en servidores propios con GPU

Ventajas Principales (Pros)

Costo marginal por token cercano a cero a saturación plena

Sin cuotas de throttling impuestas por terceros (100 % capacidad)

Control absoluto de versiones de modelos (sin retiros forzados)

Aislamiento de cómputo sin multitenencia compartida

Soporte de personalizaciones profundas y fine-tuning propio

vs

Límites y Desventajas (Contras)

Costo fijo alto 24/7 ($734 USD/mes base por g5.xlarge)

Mantenimiento continuo: parches de SO, drivers y kernels CUDA

Escalado elástico lento por tiempo de arranque en frío (Cold Start)

Inviable correr modelos gigantes de 400B+

Riesgo de capacidad ociosa si baja el tráfico de usuarios

Veredicto de Arquitectura: Justificable únicamente cuando el volumen de consultas satura la máquina

Si el tráfico es esporádico o concentrado en horas de oficina, una instancia dedicada genera un enorme desperdicio financiero

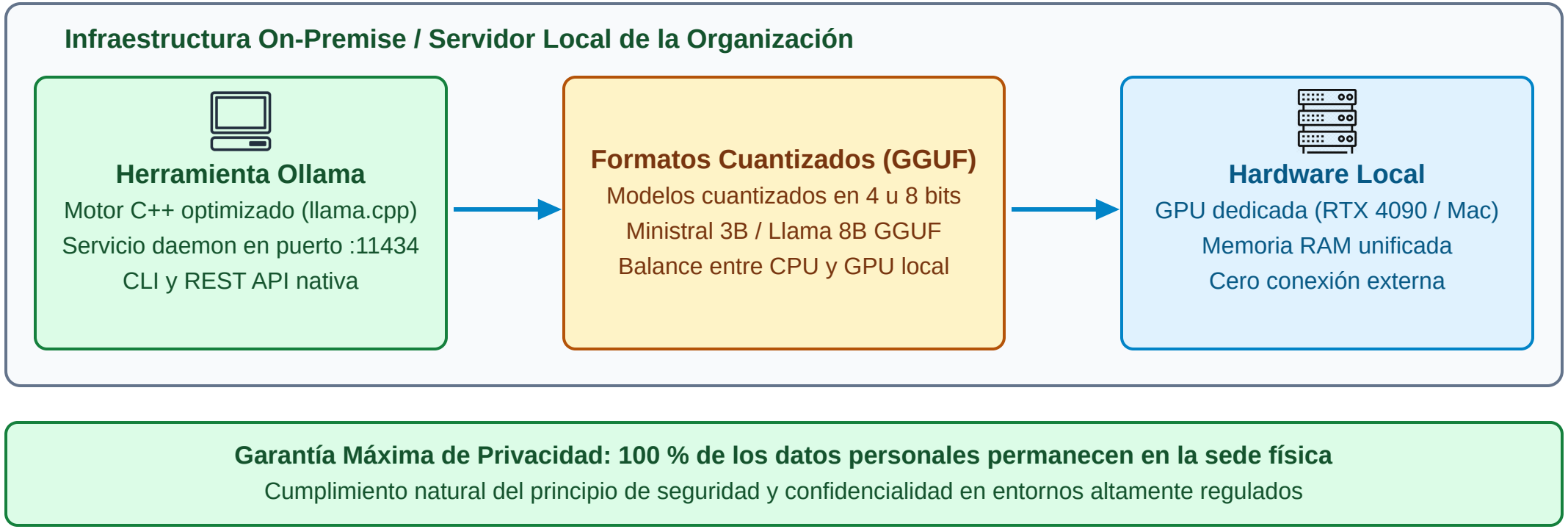
La deuda técnica de operar clústeres de GPU debe contabilizarse dentro del costo total de propiedad (TCO)

Fuente: FinOps Foundation: Managing Cloud Costs for AI Workloads · AWS Well-Architected Framework

Arquetipo 3: Inferencia local y soberana con Ollama

Máxima confidencialidad sin salida de datos para casos ultra-restringidos

Arquetipo 3: Inferencia Soberana y Local con Ollama



Garantía Máxima de Privacidad: 100 % de los datos personales permanecen en la sede física

Cumplimiento natural del principio de seguridad y confidencialidad en entornos altamente regulados

Fuente: github.com/ollama/ollama (documentación técnica oficial de ejecución local)

Fuente: Ollama Open Source Project Documentation (github.com/ollama/ollama)

Soberanía del dato y regulación en Colombia

Cumplimiento de la Ley 1581 de 2012 en arquitecturas de inteligencia artificial

Exigencias Regulatorias en Colombia (Ley 1581 de 2012 y Circular SIC)

Ley 1581 de 2012 (Habeas Data)	**Régimen General de Protección de Datos Personales** Prohíbe transferir datos a países sin niveles adecuados de seguridad (Artículo 26)
Regiones Cloud Aprobadas	**Superintendencia de Industria y Comercio (SIC)** EE.UU. (us-east-1) y Unión Europea reconocidos como destinos con estándar adecuado
Principio de Confidencialidad	**Tratamiento por Terceros y Modelos Fundacionales** Exige acuerdos contractuales que prohíban usar los datos para reentrenar modelos públicos
Consentimiento Informado	**Transparencia con el Titular del Dato** El usuario debe autorizar expresamente el procesamiento asistido por algoritmos de IA

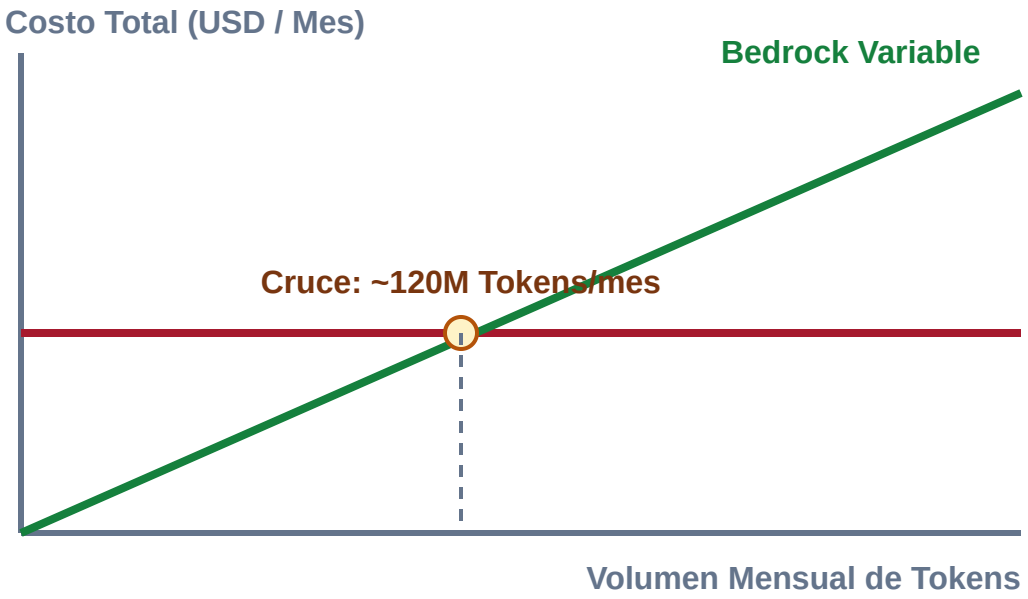
Fuente: Ley 1581 de 2012 (Congreso de Colombia) y Circulares de la Superintendencia de Industria y Comercio

Fuente: Gestor Normativo Función Pública · Ley 1581 de 2012 (Consulta: 2026-11-05 UTC)

Punto de equilibrio: ¿Cuándo conviene servidor propio?

Modelo financiero de cruce entre costos fijos de cómputo y costos variables serverless

Punto de Equilibrio: Servidor Dedicado EC2 frente a Amazon Bedrock



Zona A: Menos de 120M tokens / mes

Tráfico bajo o fluctuante

Amazon Bedrock es MUCHO más económico

Cero desperdicio de capacidad ociosa

Zona B: Más de 120M tokens / mes

Tráfico continuo y predecible 24/7

El servidor EC2 satura su capacidad

El costo marginal por token es inferior

Regla FinOps: No migrar a servidores dedicados hasta que la factura variable de la API supere el costo fijo

Incluir en el balance los costos ocultos de administración de infraestructura, ingeniería y monitoreo continuo

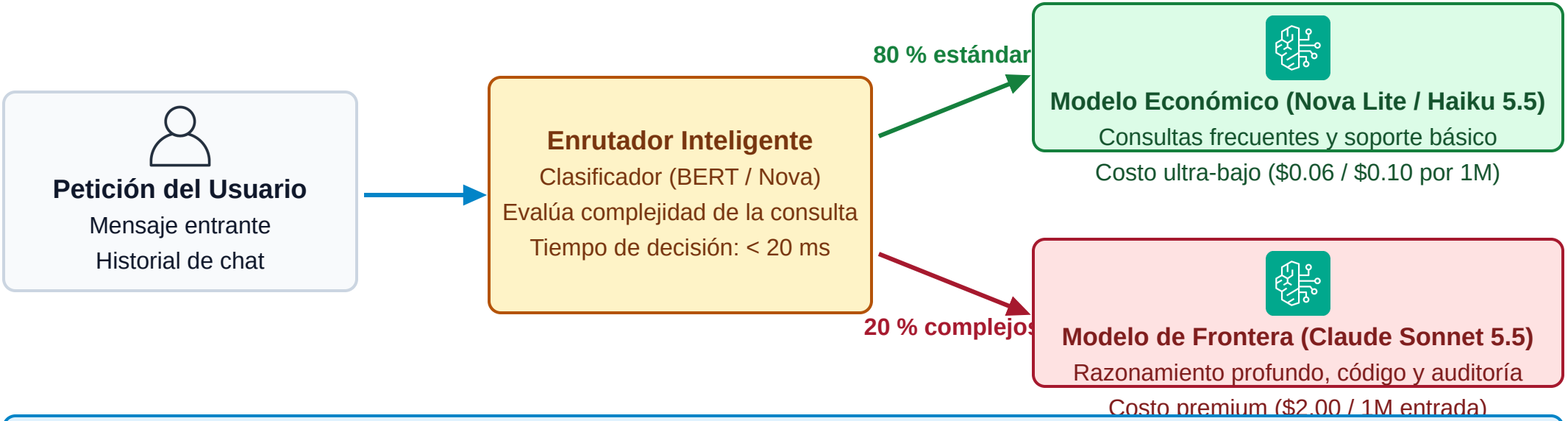
Tarifas validadas: g5.xlarge a $1.006/h ($734.38/mes) · Precios vigentes al 2026-11-05 UTC

Fuente: AWS Price List API (EC2 g5.xlarge y Bedrock us-east-1, consulta oficial 2026-11-05); cálculo matemático de arquitectura

Arquitectura híbrida: el enrutador inteligente

Segmentación de carga mediante clasificación semántica para máxima eficiencia

Arquitectura Híbrida: Enrutador Semántico de Modelos (LLM Router)



Impacto Arquitectónico Global

Mantiene la percepción de máxima calidad cognitiva atendiendo los casos difíciles con el mejor modelo

Reduce la factura mensual en más de un 60 % comparado con enviar el 100 % del tráfico al modelo de frontera

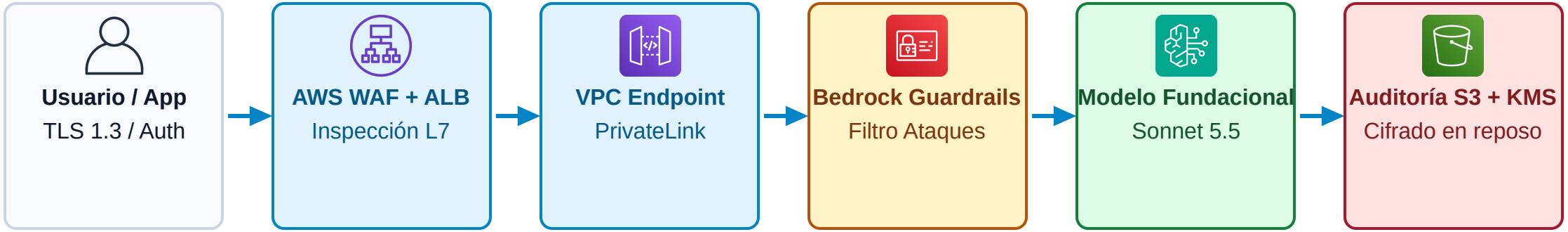
Patrón de producción estándar recomendado en el AWS Well-Architected Framework Generative AI Lens

Fuente: AWS Generative AI Lens (Architecture Patterns: Model Routing and Cascading)

Topología de referencia para IA en producción

Red privada, inspección perimetral, cifrado KMS y auditoría integral de cumplimiento

Topología Integral de Referencia Segura en Amazon Web Services



Pilares de Seguridad Implementados en la Topología

1. Aislamiento de Red: Cero exposición a internet mediante PrivateLink interno en la VPC
2. Control de Acceso: Roles de IAM de mínimo privilegio y autenticación obligatoria SigV4
3. Cifrado Extremo a Extremo: TLS en tránsito y AWS KMS con claves administradas por el cliente (CMK) en reposo
4. Filtrado: Amazon Guardrails bloquea ataques de inyección y fuga de PII

Arquitectura validada según las directrices de seguridad de ISO 27001 y SOC 2 Type II en AWS

Fuente: AWS Security Reference Architecture (SRA) · Machine Learning Security Guide

Seguridad en IA: mitigando el Prompt Injection

Defensas arquitectónicas contra la vulnerabilidad número uno del estándar OWASP

Mitigación de Inyección de Instrucciones (Vulnerabilidad OWASP LLM01:2025)

Vector de Ataque: Prompt Injection

El atacante inserta instrucciones maliciosas en el texto

Objetivo: Anular reglas ("Ignora lo previo")

Riesgo: Exfiltración de datos y abuso de tools

Puede ser directo (chat) o indirecto (PDF/RAG)

mitigado por →

Estrategias de Mitigación en Arquitectura

1. Amazon Guardrails contra Prompt Attacks
2. Delimitación con tags XML
3. Mínimo Privilegio en tokens de API
4. Human-in-the-Loop para acciones críticas

Directriz Oficial OWASP Top 10 for LLM Applications (2025/2026)

Tratar todas las entradas de usuarios y documentos externos como código no confiable y potencialmente hostil

Nunca otorgar al modelo credenciales que permitan borrar bases de datos o transferir fondos sin confirmación de usuario

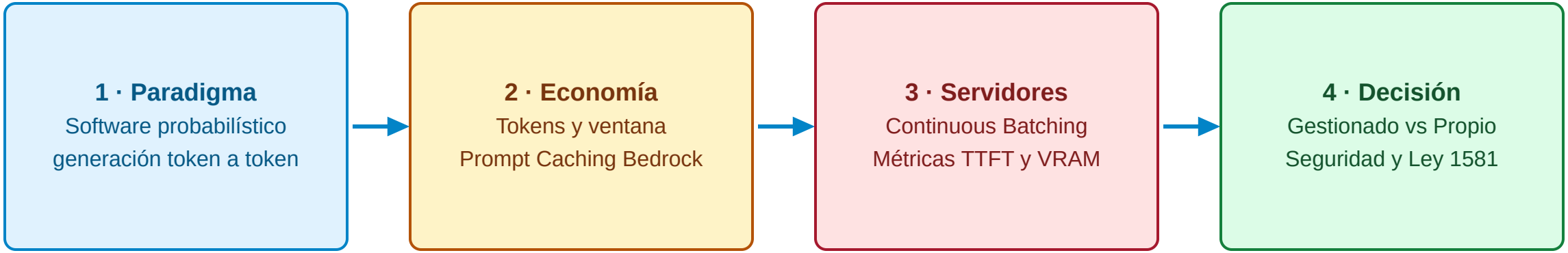
La seguridad no se resuelve con prompts ("por favor sé seguro"): se resuelve con controles arquitectónicos externos

Fuente: genai.owasp.org/llmrisk/llm01-prompt-injection/ y docs.aws.amazon.com/bedrock/latest/userguide/guardrails.html

SÍNTESIS INTEGRAL

Semana 14 en cuatro ideas fundamentales

Consolidación de conceptos de la primera semana de la Unidad 3



Los cuatro fundamentos que conectan la teoría algorítmica con la infraestructura de nube

Fuente: Síntesis pedagógica de la Semana 14 · Asignatura Arquitectura en la Nube

PRÓXIMA CLASE

Adelanto de la Semana 15: GPU y Cuantización

De la arquitectura conceptual al silicio y la compresión matemática de modelos



Semana 15: Optimización extrema de modelos, aceleradores de silicio y microarquitectura

Conclusión y lecturas recomendadas

Documentación oficial de proveedores, estándares de seguridad y marco regulatorio

AWS Bedrock Documentation	**Guías de usuario, seguridad y arquitectura de referencia** docs.aws.amazon.com/bedrock
NVIDIA Developer Architecture	**Documentación técnica de aceleradores A10G, L40S y H100** developer.nvidia.com
OWASP Top 10 for GenAI	**Estándar global de ciberseguridad para aplicaciones con LLMs** genai.owasp.org
Ley 1581 de 2012 (Colombia)	**Marco legal de protección de datos y transferencia transfronteriza** funcionpublica.gov.co

Material de estudio oficial y referencias normativas para las actividades de la Unidad 3